fortranBridge: Architectural Principles, Memory Semantics, and Verification for Fortran-to-C++ and Fortran-to-Python Translation

A Technical and Algorithmic Reference Manual

scienceStack & Antigravity Scientific Team
`https://github.com/seoym3919/fortranBridge_antigravity`

November 1, 2026

Abstract

Decades of foundational scientific simulations in astrophysics, fluid dynamics, climate modeling, and particle physics exist exclusively in legacy Fortran (FORTRAN 66/77 and early Fortran 90). Migrating this critical software into modern computational ecosystems—specifically modern C++ (C++17/20) and vectorized scientific Python (NumPy, SciPy, Numba)—is hindered by profound semantic divergences: 1-based versus 0-based array indexing, column-major versus row-major memory stride order, unthread-safe `COMMON` block global state, unstructured `GOTO` control flow, and subtle intrinsic function roundoffs. This manual presents the design, algorithms, and theoretical guarantees of **fortranBridge**, an automated translation engine designed for Google Antigravity. We formulate formal transformation rules for index normalization, memory contiguity loop transposition, `COMMON` block encapsulation, formatted I/O decompilation, dataflow intent inference, multi-file topological dependency linking, and closed-loop floating-point round-trip verification certifying numerical equivalence ($\text{Rel}\, L_2 \leq 10^{-12}$).

Contents

1 Installation and How to Use **2**
- 1.1 Installation and System Requirements . . . 2
- 1.2 How to Use: Antigravity Slash Commands . . . 2
- 1.3 Subagent Delegation Architecture . . . 3
- 1.4 How to Use: Standalone Command-Line Execution . . . 3

2 Introduction and the Fortran Migration Mandate **3**

3 Indexing Paradigms and Off-by-One Guarding **4**
- 3.1 1-Based to 0-Based Coordinate Translation . . . 4
- 3.2 Arbitrary and Negative Lower Bounds . . . 4

4 Memory Layout and Memory Contiguity Transposition **5**
- 4.1 Column-Major vs. Row-Major Storage . . . 5

5 Global State Encapsulation: COMMON and EQUIVALENCE **5**
- 5.1 COMMON Block State Encapsulation . . . 5
- 5.2 EQUIVALENCE Statement De-Aliasing . . . 6

6 Control Flow De-Spaghettification and Formatted I/O **6**
6.1 Arithmetic IF and Computed GOTO 6
6.2 Fortran FORMAT to C++20 std::format and Python f-Strings 7

7 Static Dataflow Intent Inference **7**

8 Multi-File Project Linking and Topological DAG **7**

9 Closed-Loop Numerical Equivalence Verification **7**

10 Conversion Strategy: Plan, Approval, Golden Data, Regression **8**

11 Identical Equations and Algorithm **8**

12 Summary and Command Reference **10**

1 Installation and How to Use

1.1 Installation and System Requirements

The `fortranBridge` plugin complies strictly with Agent Plugins Specification 1.0.0. It requires Python 3.9+ with standard scientific libraries (`numpy`, `scipy`) and standard C++ compilers (GCC/Clang supporting C++17 or C++20).

To validate and install the plugin globally within Google Antigravity:

```
# 1. Validate compliance strictly against Agent Plugins 1.0.0
agy plugin validate /home/seo3919/AI_skills/fortranBridge_antigravity
   --strict

# 2. Install globally in Antigravity
agy plugin install /home/seo3919/AI_skills/fortranBridge_antigravity
```

Verify active status in Antigravity:

```
agy plugin list
# - fortranbridge (enabled, 0.4.0)
#   Components: 10 skills, 2 agents, 1 rule
```

1.2 How to Use: Antigravity Slash Commands

Within the Antigravity chat interface, users interact directly with Fortran translation and auditing tools targeting directories or individual routines:

- `/translate-fortran [directory] --target <cpp|python>`: **Master codebase-wide translation command**. Ingests the directory, resolves multi-file dependency DAGs, and generates a structured C++ project (with `CMakeLists.txt`) or Python package (with `__init__.py`).
- `/fortran-to-cpp [directory]`: Generates modern C++17/20 with contiguous cache memory layouts, `std::span`, and build files.
- `/fortran-to-python [directory]`: Generates vectorized NumPy package with `@dataclass` state models.
- `/audit-indexing [directory]`: Audits 1-based to 0-based subscript translations, array bounds, and memory layout strides across all files.
- `/encapsulate-common [directory]`: Converts unthread-safe `COMMON` blocks across the codebase into structured containers.
- `/restructure-goto <file>`: De-spaghettifies backward loops, arithmetic `IF`s, and computed `GOTO`s.
- `/verify-translation --ref FORTRAN_OUT --test TRANSLATED_OUT`: Runs closed-loop numerical equivalence testing; exits non-zero unless both relative L_2 and relative L_∞ are $\leq 10^{-12}$ (exit 2: nothing compared; exit 3: verdict withheld because the tolerance was loosened after a failure on unchanged files; every run is appended to a hash-chained evidence log bound to the SHA-256 of both files).
- `/decompile-format <file>`: Decompiles Fortran edit descriptors into C++20 `std::format` and Python f-strings.
- `/link-project <dir>`: Analyzes multi-file `MODULE`/`USE` dependencies and generates `CMakeLists.txt`.
- `/infer-intents <dir>`: Performs dataflow analysis to infer `const T&` in C++ and return tuples in Python.

1.3 Subagent Delegation Architecture

Complex tasks are automatically routed to specialized subagents:

- `fortran-translator` (`Model: "pro"` — `claude-opus-5-5-high` / `claude-sonnet-5-5-high` / `gemini-4-argon-high`): Lead translation architect. Manages codebase traversal, multi-file AST decomposition, C++17/20 and vectorized Python modernization, and project packaging.
- `indexing-auditor` (`Model: "pro"` — `claude-opus-5-5-high`): Numerical and mathematical indexing auditor.

1.4 How to Use: Standalone Command-Line Execution

For offline environments or zero-token local processing, all tools operate via direct CLI commands:

```
# 1. Modernize entire Fortran codebase directory to modern C++ with CMakeLists.txt
python3 scripts/fortran_bridge_cli.py /path/to/fortran_repo --target cpp

# 2. Modernize entire Fortran codebase directory to vectorized Python with __init__.py
python3 scripts/fortran_bridge_cli.py /path/to/fortran_repo --target python --out-dir modernized_py

# 3. Modernize an individual Fortran source file
python3 scripts/fortran_bridge_cli.py orbit.f --target cpp --out orbit.cpp

# 4. Analyze multi-file project dependencies and build topological compilation graph
python3 scripts/project_linker.py /path/to/fortran_project

# 5. Run numerical equivalence self-test suite
python3 scripts/translation_verifier.py --self-test      # checks the verifier itself, not a translation
python3 scripts/translation_verifier.py --ref fortran.out --test translated.out   # verifies a translation

# 6. Run the 114-test unit suite (parser semantics, verifier, evidence log, NumPy/C++ equivalence, CLI flags & Python 3.10 portability)
python3 -B -m unittest discover -s tests -v
```

2 Introduction and the Fortran Migration Mandate

Fortran has served as the backbone of high-performance scientific computing since its introduction by John Backus in 1957. Production codes built over seven decades contain millions of lines of rigorously verified numerical algorithms. However, modern scientific software infrastructure demands features absent in legacy Fortran:

1. **Object-Oriented and Functional Modularity**: Safe state encapsulation, thread safety, and dependency injection.
2. **GPU and Heterogeneous Acceleration**: CUDA, SYCL, Kokkos, and JAX/PyTorch tensor primitives.

3. **Modern Memory Semantics**: Zero-overhead abstractions like `std::span`, `std::vector`, and contiguous NumPy arrays.

4. **Interoperability and Ecosystem Integration**: REST APIs, interactive Jupyter environments, and machine learning libraries.

Manual translation of legacy Fortran into C++ or Python is notoriously error-prone. Even experienced computational scientists frequently introduce subtle off-by-one subscript errors, inadvertent cache-thrashing memory strides, and uninitialized aliasing defects. **fortranBridge** provides an automated, mathematically rigorous translation pipeline designed to solve these systemic issues.

3 Indexing Paradigms and Off-by-One Guarding

3.1 1-Based to 0-Based Coordinate Translation

The most prevalent source of catastrophic translation failure is the disparity between Fortran's default 1-based indexing and C++/Python's 0-based indexing:

$$\text{Fortran: } A(1), A(2), \ldots, A(N) \quad \longleftrightarrow \quad \text{C++/Python: } a[0], a[1], \ldots, a[N-1]. \tag{1}$$

When translating loops of the canonical form:

```
      DO 10 I = 1, N
         A(I) = B(I) + C(I)
 10   CONTINUE
```

`fortranBridge` applies unified index coordinate transformation:

$$i_{\text{C++}} = i_{\text{Fortran}} - 1, \quad i_{\text{C++}} \in [0, N-1]. \tag{2}$$

In C++, this yields:

```
for (int i = 0; i < n; ++i) {
    a[i] = b[i] + c[i];
}
```

And in Python, when loop vectorization is not possible:

```
for i in range(n):
    a[i] = b[i] + c[i]
```

Where vectorization is applicable, the entire loop reduces to a single vectorized statement:

```
a = b + c
```

3.2 Arbitrary and Negative Lower Bounds

Fortran permits arbitrary array dimensions:

$$\texttt{DIMENSION A(L_1:U_1, L_2:U_2)}. \tag{3}$$

For an array declared as `DIMENSION A(-5:5, 0:10)`, the index mapping for element $A(i, j)$ into 0-based C++ coordinates is:

$$i_{\text{C++}} = i - L_1 = i - (-5) = i + 5, \quad j_{\text{C++}} = j - L_2 = j - 0 = j. \tag{4}$$

The `indexing-auditor` agent formally verifies that all shifted array subscripts are non-negative and strictly bounded by the allocated dimension extent $U_k - L_k + 1$.

4 Memory Layout and Memory Contiguity Transposition

4.1 Column-Major vs. Row-Major Storage

Consider a 2D matrix $A \in \mathbb{R}^{M \times N}$. The linear memory offset of element $A(i, j)$ differs fundamentally across programming paradigms:

$$\text{Fortran (Column-Major):} \quad \text{offset}(i, j) = (i - 1) + (j - 1) \cdot M, \tag{5}$$

$$\text{C / C++ (Row-Major):} \quad \text{offset}(i, j) = i \cdot N + j. \tag{6}$$

In Fortran, the first index i varies fastest in linear memory. Consequently, canonical Fortran algorithms nest loops with I in the innermost position:

```
      DO 20 J = 1, M
         DO 10 I = 1, N
            A(I, J) = B(I, J) * C(I, J)
 10      CONTINUE
 20   CONTINUE
```

Proposition 4.1 (Cache Line Contiguity Principle)**.** If a Fortran nested loop is translated naively to C++ without loop or array transposition:

```
// Cache-Destructive Naive Translation (Stride = N doubles per
   iteration):
for (int j = 0; j < m; ++j) {
    for (int i = 0; i < n; ++i) {
        a[i][j] = b[i][j] * c[i][j]; // Cache miss on every memory
           fetch!
    }
}
```

Memory access jumps across rows by stride N, loading a 64-byte cache line but consuming only a single 8-byte double before jumping, destroying L1/L2 data cache hit rates.

`fortranBridge` addresses this by applying **Loop Stride Inversion**:

```
// Cache-Optimal Transposed C++ Translation:
for (int i = 0; i < n; ++i) {
    for (int j = 0; j < m; ++j) {
        a[i][j] = b[i][j] * c[i][j]; // Sequential, SIMD-vectorizable
           memory loads
    }
}
```

Or, when interfacing with external Fortran-ordered libraries, explicitly retaining column-major ordering via `Eigen::Matrix<double, Dynamic, Dynamic, ColMajor>` or NumPy's `order='F'`.

5 Global State Encapsulation: COMMON and EQUIVALENCE

5.1 COMMON Block State Encapsulation

Legacy Fortran codes share variables between subroutines using global memory layouts known as `COMMON` blocks:

```
      SUBROUTINE STEP_ORBIT
      COMMON /ORBIT_DATA/ ECC, SEMI, INC, POS(3), VEL(3)
      ...
      END
```

In modern concurrent C++ and multi-threaded Python, global mutable state leads to catastrophic data races and prevents re-entrant simulation instances.

`fortranBridge` translates each named `COMMON /NAME/` block into a dedicated state structure:

```cpp
// Modern C++ Encapsulated Struct
struct OrbitDataBlock {
    double ecc{0.0};
    double semi{0.0};
    double inc{0.0};
    std::array<double, 3> pos{};
    std::array<double, 3> vel{};
};

// Subroutine accepts state reference
void step_orbit(OrbitDataBlock& orbit_state);
```

In Python, `fortranBridge` translates the block into a typed `@dataclass`:

```python
from dataclasses import dataclass, field
import numpy as np

@dataclass
class OrbitDataBlock:
    ecc: float = 0.0
    semi: float = 0.0
    inc: float = 0.0
    pos: np.ndarray = field(default_factory=lambda: np.zeros(3))
    vel: np.ndarray = field(default_factory=lambda: np.zeros(3))
```

5.2 EQUIVALENCE Statement De-Aliasing

In FORTRAN 77, `EQUIVALENCE` statements force variables to share physical memory locations:

```
      DIMENSION VEC(3)
      EQUIVALENCE (VEC(1), X), (VEC(2), Y), (VEC(3), Z)
```

The `equivalence_analyzer` engine identifies whether the aliasing represents:

- **Scalar-to-Vector Overlay**: Decompiled to C++ references (`double& x = vec[0];`) or Python tuple unpacking.
- **1D Flat Buffer to 2D Matrix Reshaping**: Decompiled to zero-copy `std::span<double>` or NumPy strided memory views (`mat = flat.view().reshape((M, N), order='F')`).
- **Type Punning**: Decompiled to C++20 `std::bit_cast` or explicit unions.

6 Control Flow De-Spaghettification and Formatted I/O

6.1 Arithmetic IF and Computed GOTO

$$\texttt{IF (E) L1, L2, L3} \iff \begin{cases} \text{Jump to } L_1 & \text{if } E < 0, \\ \text{Jump to } L_2 & \text{if } E = 0, \\ \text{Jump to } L_3 & \text{if } E > 0. \end{cases} \tag{7}$$

This is decompiled into standard C++ `if (e < 0) ... else if (e == 0) ... else ...` blocks. Similarly, `GOTO (L1, L2, L3) I` is decompiled into a C++ `switch(i)` or Python `match/case` construct.

6.2 Fortran FORMAT to C++20 std::format and Python f-Strings

Fortran formatted I/O uses cryptic descriptor strings:

```
100   FORMAT(1X, 'ENERGY = ', 1P, E15.7, 2X, 'STEP = ', I6)
      WRITE(6, 100) ENERGY, ISTEP
```

The `format_decompiler` engine maps descriptors to modern typesafe string formatters:

C++20: `std::cout << std::format(" ENERGY = {:15.7e} STEP = {:6d}\n", energy, istep);` (8)

Python: `print(f" ENERGY = {energy:15.7e} STEP = {istep:6d}")` (9)

7 Static Dataflow Intent Inference

Because FORTRAN 77 lacked argument intent keywords, all arguments were passed as mutable pointers. The `intent_analyzer` engine constructs a definition-use (def-use) dataflow graph for each routine:

- **INTENT(IN)**: Variables only read in expressions $\longrightarrow$ `const T&` or pass-by-value in C++; standard input parameter in Python.
- **INTENT(OUT)**: Variables only written to $\longrightarrow$ `T&` in C++; extracted from caller arguments and returned as clean tuple in Python (`return out1, out2`).
- **INTENT(INOUT)**: Variables both read and updated $\longrightarrow$ mutable `T&` in C++; in-place array modification in Python.

8 Multi-File Project Linking and Topological DAG

Production scientific simulations consist of numerous interdependent files. The `project_linker` engine scans codebases, identifying:

$$\text{Dependency Edges: } \mathcal{E} = \{(F_i, F_j) \mid F_i \text{ uses MODULE or calls routine defined in } F_j\}. \tag{10}$$

It executes a topological sort on the resulting Directed Acyclic Graph (DAG) to determine the strict compilation order and emits modular C++ headers (`.hpp`) with `#pragma once` alongside a turnkey `CMakeLists.txt`.

9 Closed-Loop Numerical Equivalence Verification

The `translation_verifier` module compares the output files of the legacy Fortran program and of the translation, produced for the same inputs, and `f2py_harness` drives such comparisons across parameter phase space. A comparison with a different number of values, no values, or any NaN or infinity always fails. The printed diagnostics are

$$\text{Rel } L_2 = \frac{\|\mathbf{y}_{\text{Fortran}} - \mathbf{y}_{\text{Port}}\|_2}{\|\mathbf{y}_{\text{Fortran}}\|_2 + \varepsilon}, \tag{11}$$

$$\text{Rel } L_\infty = \frac{\max |\mathbf{y}_{\text{Fortran}} - \mathbf{y}_{\text{Port}}|}{\max |\mathbf{y}_{\text{Fortran}}| + \varepsilon}. \tag{12}$$

They are not the pass rule. The verifier exits 0 only when every element satisfies

$$|y_{\text{Port},i} - y_{\text{Fortran},i}| \le atol + rtol\,|y_{\text{Fortran},i}| \tag{13}$$

with float64 defaults $rtol = 10^{-12}$ and $atol = 0$, the IEEE-754 ULP distance is inside that budget, and +0 versus −0 fails. Array shape and float32 versus float64 width must match. Comparison is by index, so the same logical values stored in C or Fortran order agree.

A complete port is `port_completeness.py --target python|cpp`, claimed only when it exits 0. Every original equation must be translated. Every procedure other than a `PROGRAM` must have a bitwise unit test, and the latest verifier record must have passed with $atol = 0$ and max_ulp = 0 on checker version 2 or newer. A `PROGRAM` is emitted as `main`. If the values differ, the translation is investigated. The tolerance is not raised.

10 Conversion Strategy: Plan, Approval, Golden Data, Regression

A whole codebase is converted in a fixed order, and the tools refuse to skip a step.

1. **Plan.** `conversion_plan.py SOURCES --target python|cpp` lists every function and module in `FORTRAN_FUNCTION_LIST.md`, runs the Fortran under `gprof` to see how they link, and writes `PLAN.md`: the order of conversion (callees first), the rules, and the golden data table.

2. **Approval.** Nothing is converted or tested until a human approves that exact `PLAN.md` by typing `APPROVE PLAN <code>` as their own message. A hook reads the message. Any edit to `PLAN.md` voids the approval.

3. **Golden data.** `regression_test.py record` stores the Fortran results as bytes before conversion, so a later change cannot redefine the expected answer. Every function must be covered, with nominal and corner cases and more than 10 distinct whole-program cases.

4. **Convert one function or module at a time**, in plan order. After each function comes a unit test (`unit_compare.py` runs both sides itself and compares raw bytes) and at least one regression test (`regression_test.py run --after`). A function is refused until everything earlier in the plan has passed both.

5. **Final regression.** `regression_test.py run --final` runs every case.

Byte identity is the rule throughout. Agreement within machine precision, with relative error at most the machine epsilon of the data (2.22×10^{-16} for float64, 1.19×10^{-7} for float32) and at most 2 ULP, is accepted only for what the plan shows reaching a library such as `sin`, `exp` or `MATMUL`, where Fortran and the target need not share the last bit. `sqrt` is not on that list: IEEE 754 fixes it, so a difference there is a bug.

The approval and the evidence are plain files, so these steps make approval a human step in normal use. They are not a defence against something writing those files directly.

11 Identical Equations and Algorithm

Matching output shows that some numbers agree on some inputs. A translation is therefore checked three ways, and a claim needs all of them; each catches what the others cannot.

Check	Compares	Catches
`identity_checker`	gfortran's own parse tree with the translation (Python through `ast`, C++ through clang), statement by statement; `GO TO` programs by walking both control-flow graphs in lockstep	reassociated or reordered arithmetic, changed loops, bounds, branches, kinds, literals, declarations, returned results
`trace_checker`	the bits of every assignment, in order, between instrumented copies of both programs; every statement must execute	a helper that matches its table entry but computes something else; a statement no input reached
`translation_verifier`	the printed results	values that are printed but never assigned

The checkers do not import the translator, so a translator bug cannot hide itself. The one shared piece of knowledge is a rewrite table in `identity_checker`: what each construct of the translation means in Fortran. A construct that is not in the table is a failure, never a pass. The instrumented Fortran must print exactly what the original prints, or the run is rejected. Exit status is 0 for identical, 1 for different and 2 when nothing was established; exit 2 is never a pass.

Verdicts. The final message of a turn that states a result ends with one `VERDICT:` line. A Stop hook (`claim_guard.py`) checks that line, and any sentence claiming equivalence or a complete port, against the evidence log `.verification/log.jsonl`, and blocks the stop when the evidence is missing, stale or looser than the rules allow. It repeats the static check itself instead of trusting the log.

Verdict	Needs
`equivalent`	a plan, a passing identity check and a bitwise compare ($atol = 0$, max_ulp $= 0$); for a Python port also a bitwise trace
`identical algorithm`	a plan that shows a Fortran-specific library, identity and (Python) trace with `--machine-precision`, and a compare within machine precision. The equations and the algorithm are the same and a library differs in the last bits; it is never reported as bitwise
`complete`	as `equivalent`, plus `port_completeness.py` on the same files and a fresh passing unit test for every function of the plan

Limits.

- Reading C++ needs clang (`clang++`, or `FORTRANBRIDGE_CXX`); g++ cannot produce this AST. The trace reads Python only, so a C++ port has the static check, the unit tests and the regression, without a statement-level trace.
- "Identical" means agreement with a `-O0 -fwrapv` build of the source. Another optimisation level can print different numbers: 4 of 43 fixtures differ between `-O0` and `-O2`.
- This is structural identity of the program given and bitwise agreement on the inputs run, not a proof about the translator.
- `MATMUL` calls libgfortran's own routine. Without the library it falls back to a fixed-order sum, measured at up to 18 ULP from gfortran, which is more than machine precision, so that program earns neither `equivalent` nor `identical algorithm`.

12 Summary and Command Reference

Listing 1: Running fortranBridge CLI Tools

```
# Modernize an entire Fortran codebase directory to C++17/20 with CMakeLists.txt
python3 scripts/fortran_bridge_cli.py /path/to/fortran_repo --target cpp

# Modernize an entire Fortran codebase directory to a vectorized Python package
python3 scripts/fortran_bridge_cli.py /path/to/fortran_repo --target python --out-dir modernized_py

# Translate a single Fortran source file with an audit report
python3 scripts/fortran_bridge_cli.py solver.f --target python --out solver.py --report audit.md

# Run closed-loop numerical equivalence test suite
python3 scripts/translation_verifier.py --self-test      # checks the verifier itself, not a translation
python3 scripts/translation_verifier.py --ref fortran.out --test translated.out \
    --ref-source legacy.f --test-source port.py --inputs inputs.txt
python3 scripts/port_completeness.py --original legacy.f --translated port.py \
    --target python --unit-tests tests --evidence-log .verification/log.jsonl
python3 scripts/f2py_harness.py --self-test

# The conversion strategy, in order
python3 scripts/conversion_plan.py legacy.f --target python    # FORTRAN_FUNCTION_LIST.md and PLAN.md; then a human types APPROVE PLAN <code>
python3 scripts/regression_test.py record                      # golden data from the Fortran, before converting
python3 scripts/unit_compare.py --function NAME --fortran-driver DRIVER --target-test TEST \
    --target-command COMMAND --translated port.py              # after each function
python3 scripts/regression_test.py run --after NAME --translated port.py
python3 scripts/regression_test.py run --final --translated port.py

# Same equations and algorithm
python3 scripts/identity_checker.py --original legacy.f --translated port.py
python3 scripts/trace_checker.py --original legacy.f --translated port.py [--stdin inputs.txt] [--machine-precision]
```